INVERTIS UNIVERSITY, BAREILLY

B.Tech CSE - Third Year

Applied Data Structures & Algorithms Lab (BCS553)

Lab Practical File - Assignments 1 to 15

Name: _____

Roll Number: _____

INDEX

No.	Experiment Title
1	Reading Array Values
2	Largest, Smallest and Count
3	Bubble Sort
4	Selection Sort
5	Linear Search
6	Binary Search on a Sorted Array
7	Practice Set
8	Classes and Objects
9	Constructors
10	Parent and Child Classes
11	Method Overriding
12	Recursive Factorial, Sum and Power
13	Recursion on Arrays
14	Sorted Check, Search and Fibonacci
15	Subsets and Arrangements

EXPERIMENT NO. 1

1. Experiment Title

Reading Array Values

2. Aim

To read values from an array by index and by a loop.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

An array stores a fixed number of values of the same type in contiguous memory. Each element is accessed with a zero-based index: the first element is a[0] and the last is a[a.length - 1]. A for loop from 0 to a.length - 1 visits every element, and the same loop can accumulate a running total.

5. Algorithm / Procedure

Step 1: Create the array marks = {45, 67, 80, 52, 90}.
Step 2: Task 1: print marks[0] as First and marks[marks.length - 1] as Last.
Step 3: Task 2: loop i from 0 to length - 1 and print marks[i] on its own line.
Step 4: Task 3: set sum = 0; loop over all elements adding each to sum; print sum.

Time / Space Complexity: Task 1: O(1) time. Tasks 2 and 3: O(n) time. Space O(1) extra in all tasks.

6. Program / Queries / Commands

Task 1: Print first and last value

```
public class Main {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        System.out.println("First: " + marks[0]);
        System.out.println("Last: " + marks[marks.length - 1]);
    }
}
```

Task 2: Print every value with a for loop

```
public class Main {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        for (int i = 0; i < marks.length; i++) {
            System.out.println(marks[i]);
        }
    }
}
```

Task 3: Sum of all values

```
public class Main {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        int sum = 0;
        for (int i = 0; i < marks.length; i++) {
            sum += marks[i];
        }
        System.out.println("Sum: " + sum);
    }
}
```

7. Input

Task 1: none (array is fixed in code)
Task 2: none (array is fixed in code)
Task 3: none (array is fixed in code)

8. Output

Task 1 output (actual run)

```
$ java Main.java
First: 45
Last: 90
```

Task 2 output (actual run)

```
$ java Main.java
45
67
80
52
90
```

Task 3 output (actual run)

```
$ java Main.java
Sum: 334
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To read values from an array by index and by a loop.

10. Viva-Voce Questions

Q1. What is the index of the first element of a Java array?
Ans: 0.

Q2. How do you get the number of elements in an array?
Ans: Using the length field, e.g. marks.length (no parentheses).

Q3. What happens if you access marks[5] here?
Ans: An ArrayIndexOutOfBoundsException is thrown, since valid indexes are 0 to 4.

Q4. Can an array change size after creation?
Ans: No, its size is fixed; a new array must be created.

EXPERIMENT NO. 2

1. Experiment Title

Largest, Smallest and Count

2. Aim

To find the largest and smallest value in an array and to count values that pass a test.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

Finding the maximum or minimum needs one pass: assume the first element is the answer, then compare every other element and update the answer when a better one is found. Counting works the same way, with an if condition that increments a counter when the test (value > 60) is true.

5. Algorithm / Procedure

Step 1: Create the array marks.
Step 2: Largest: max = marks[0]; for each i from 1, if marks[i] > max then max = marks[i]; print max.
Step 3: Smallest: min = marks[0]; for each i from 1, if marks[i] < min then min = marks[i]; print min.
Step 4: Count: count = 0; for each value, if value > 60 then count++; print count.

Time / Space Complexity: All tasks: O(n) time, O(1) extra space.

6. Program / Queries / Commands

Task 1: Largest value

```
public class Main {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        int max = marks[0];
        for (int i = 1; i < marks.length; i++) {
            if (marks[i] > max) {
                max = marks[i];
            }
        }
        System.out.println("Largest: " + max);
    }
}
```

Task 2: Smallest value

```
public class Main {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        int min = marks[0];
        for (int i = 1; i < marks.length; i++) {
            if (marks[i] < min) {
                min = marks[i];
            }
        }
        System.out.println("Smallest: " + min);
    }
}
```

Task 3: Count values greater than 60

```
public class Main {
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        int count = 0;
        for (int i = 0; i < marks.length; i++) {
            if (marks[i] > 60) {
                count++;
            }
        }
        System.out.println("Count: " + count);
    }
}
```

7. Input

Task 1: none (array is fixed in code)
Task 2: none (array is fixed in code)
Task 3: none (array is fixed in code)

8. Output

Task 1 output (actual run)

```
$ java Main.java
Largest: 90
```

Task 2 output (actual run)

```
$ java Main.java
Smallest: 45
```

Task 3 output (actual run)

```
$ java Main.java
Count: 3
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To find the largest and smallest value in an array and to count values that pass a test.

10. Viva-Voce Questions

Q1. Why start max with marks[0] instead of 0?
Ans: Starting with 0 fails for arrays of all-negative numbers; marks[0] is always a valid candidate.

Q2. What is the time complexity of finding the maximum?
Ans: O(n), as every element is checked once.

Q3. How would you find both max and min in one loop?
Ans: Keep two variables and update both in the same pass.

Q4. What does the if condition do in the count task?
Ans: It filters values so only those greater than 60 increase the counter.

EXPERIMENT NO. 3

1. Experiment Title

Bubble Sort

2. Aim

To sort an array by comparing neighbouring values and swapping them when they are out of order.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

Bubble sort repeatedly compares adjacent elements and swaps them if the left one is larger. After each pass the largest remaining value 'bubbles' to the end of the unsorted part. n - 1 passes sort an array of n elements. It is stable and sorts in place.

5. Algorithm / Procedure

Step 1: Start with the array a of length n.
Step 2: Outer loop: for i = 0 to n - 2 (one pass per iteration).
Step 3: Inner loop: for j = 0 to n - 2 - i; if a[j] > a[j + 1], swap them using a temporary variable.
Step 4: Task 1 performs only one pass (the inner loop once).
Step 5: After all passes the array is sorted; print each value on its own line.

Time / Space Complexity: Worst and average: O(n^2) time; best case O(n) only with an early-exit flag (not used here). Space O(1).

6. Program / Queries / Commands

Task 1: One pass of bubble sort on {5, 1, 4, 2, 8}

```
public class Main {
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};
        for (int j = 0; j < a.length - 1; j++) {
            if (a[j] > a[j + 1]) {
                int t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
        }
        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Task 2: Full bubble sort on {5, 1, 4, 2, 8}

```
public class Main {
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};
        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
            }
        }
        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Task 3: Bubble sort on {9, 7, 3}

```
public class Main {
    public static void main(String[] args) {
        int[] a = {9, 7, 3};
        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
            }
        }
        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

7. Input

Task 1: array {5, 1, 4, 2, 8}
Task 2: array {5, 1, 4, 2, 8}
Task 3: array {9, 7, 3}

8. Output

Task 1 output (actual run)

```
$ java Main.java
1
4
2
5
8
```

Task 2 output (actual run)

```
$ java Main.java
1
2
4
5
8
```

Task 3 output (actual run)

```
$ java Main.java
3
7
9
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To sort an array by comparing neighbouring values and swapping them when they are out of order.

10. Viva-Voce Questions

Q1. Why is it called bubble sort?
Ans: Large values move ('bubble') towards the end of the array after each pass.

Q2. What is the worst-case time complexity?
Ans: O(n^2).

Q3. Is bubble sort stable?
Ans: Yes, equal elements keep their relative order because only strictly larger values are swapped.

Q4. Why does the inner loop run to length - 1 - i?
Ans: The last i elements are already in their final place, so they need not be compared again.

EXPERIMENT NO. 4

1. Experiment Title

Selection Sort

2. Aim

To sort an array by finding the smallest value in the unsorted part and moving it to the front.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

Selection sort splits the array into a sorted left part and an unsorted right part. In each step it finds the minimum of the unsorted part and swaps it with the first unsorted element. It makes at most n - 1 swaps, which is fewer than bubble sort, but always does O(n^2) comparisons.

5. Algorithm / Procedure

Step 1: Task 1: set minIdx = 0; for i = 1 to n - 1, if a[i] < a[minIdx] set minIdx = i; print minIdx.
Step 2: Task 2: for i = 0 to n - 2: set minIdx = i; scan j = i + 1 to n - 1 for a smaller value; swap a[i] and a[minIdx].
Step 3: Task 3: perform only the i = 0 step of Task 2 and print the array.
Step 4: Print each value of the array on its own line.

Time / Space Complexity: Time O(n^2) in best, average and worst cases; at most n - 1 swaps. Space O(1).

6. Program / Queries / Commands

Task 1: Index of the smallest value in {64, 25, 12, 22, 11}

```
public class Main {
    public static void main(String[] args) {
        int[] a = {64, 25, 12, 22, 11};
        int minIdx = 0;
        for (int i = 1; i < a.length; i++) {
            if (a[i] < a[minIdx]) {
                minIdx = i;
            }
        }
        System.out.println(minIdx);
    }
}
```

Task 2: Full selection sort

```
public class Main {
    public static void main(String[] args) {
        int[] a = {64, 25, 12, 22, 11};
        for (int i = 0; i < a.length - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < a.length; j++) {
                if (a[j] < a[minIdx]) {
                    minIdx = j;
                }
            }
            int t = a[i];
            a[i] = a[minIdx];
            a[minIdx] = t;
        }
        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Task 3: First step of selection sort only

```
public class Main {
    public static void main(String[] args) {
        int[] a = {64, 25, 12, 22, 11};
        int minIdx = 0;
        for (int i = 1; i < a.length; i++) {
            if (a[i] < a[minIdx]) {
                minIdx = i;
            }
        }
        int t = a[0];
        a[0] = a[minIdx];
        a[minIdx] = t;
        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

7. Input

Task 1: array {64, 25, 12, 22, 11}
Task 2: array {64, 25, 12, 22, 11}
Task 3: array {64, 25, 12, 22, 11}

8. Output

Task 1 output (actual run)

```
$ java Main.java
4
```

Task 2 output (actual run)

```
$ java Main.java
11
12
22
25
64
```

Task 3 output (actual run)

```
$ java Main.java
11
25
12
22
64
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To sort an array by finding the smallest value in the unsorted part and moving it to the front.

10. Viva-Voce Questions

Q1. What does selection sort select in each step?
Ans: The minimum element of the unsorted part.

Q2. How many swaps does selection sort make at most?
Ans: n - 1 swaps.

Q3. Is selection sort stable?
Ans: No, the long-distance swap can change the order of equal elements.

Q4. What is its best-case time complexity?
Ans: O(n^2); it still scans the whole unsorted part every time.

EXPERIMENT NO. 5

1. Experiment Title

Linear Search

2. Aim

To search an array one value at a time and report the index where a value is found.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

Linear (sequential) search checks each element from the start until the key is found or the array ends. It works on unsorted data. It returns the index of the first match, or -1 when the key is absent. The same scan, with a counter instead of an early return, counts occurrences of a value.

5. Algorithm / Procedure

Step 1: For i = 0 to n - 1: if a[i] == key, return i.
Step 2: If the loop ends without a match, return -1.
Step 3: Tasks 1 and 2: call linearSearch with key 80 and key 100 and print the result.
Step 4: Task 3: set count = 0; for each element equal to 52 increment count; print count.

Time / Space Complexity: Time O(n) worst case, O(1) best case. Space O(1).

6. Program / Queries / Commands

Task 1: Search for 80 in {45, 67, 80, 52, 90}

```
public class Main {
    static int linearSearch(int[] a, int key) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == key) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(linearSearch(a, 80));
    }
}
```

Task 2: Search for 100 in {45, 67, 80, 52, 90}

```
public class Main {
    static int linearSearch(int[] a, int key) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == key) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(linearSearch(a, 100));
    }
}
```

Task 3: Count occurrences of 52 in {52, 10, 52, 7, 52}

```
public class Main {
    public static void main(String[] args) {
        int[] a = {52, 10, 52, 7, 52};
        int count = 0;
        for (int i = 0; i < a.length; i++) {
            if (a[i] == 52) {
                count++;
            }
        }
        System.out.println("Count: " + count);
    }
}
```

7. Input

Task 1: array {45, 67, 80, 52, 90}, key = 80
Task 2: array {45, 67, 80, 52, 90}, key = 100
Task 3: array {52, 10, 52, 7, 52}

8. Output

Task 1 output (actual run)

```
$ java Main.java
2
```

Task 2 output (actual run)

```
$ java Main.java
-1
```

Task 3 output (actual run)

```
$ java Main.java
Count: 3
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To search an array one value at a time and report the index where a value is found.

10. Viva-Voce Questions

Q1. Does linear search need a sorted array?
Ans: No.

Q2. What is its worst-case complexity?
Ans: O(n), when the key is last or absent.

Q3. Why return -1 for 'not found'?
Ans: -1 can never be a valid array index.

Q4. How does searching differ from counting?
Ans: Searching stops at the first match; counting must scan the whole array.

EXPERIMENT NO. 6

1. Experiment Title

Binary Search on a Sorted Array

2. Aim

To search a sorted array by checking the middle value and discarding half of the array each time.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

Binary search requires a sorted array. It compares the key with the middle element: if equal it is found; if the key is smaller, the search continues in the left half; otherwise in the right half. Each step halves the search range, so at most about log2(n) comparisons are needed.

5. Algorithm / Procedure

Step 1: Set low = 0 and high = n - 1.
Step 2: While low <= high: mid = low + (high - low) / 2.
Step 3: If a[mid] == key return mid; if a[mid] < key set low = mid + 1; else set high = mid - 1.
Step 4: If the loop ends, return -1.
Step 5: Call the method with keys 40, 35 and 10 on {10, 20, 30, 40, 50}.

Time / Space Complexity: Time O(log n) worst case, O(1) best case. Space O(1) (iterative).

6. Program / Queries / Commands

Task 1: Search for 40

```
public class Main {
    static int binarySearch(int[] a, int key) {
        int low = 0, high = a.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (a[mid] == key) {
                return mid;
            } else if (a[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 40));
    }
}
```

Task 2: Search for 35 (absent)

```
public class Main {
    static int binarySearch(int[] a, int key) {
        int low = 0, high = a.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (a[mid] == key) {
                return mid;
            } else if (a[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 35));
    }
}
```

Task 3: Search for 10 (first element)

```
public class Main {
    static int binarySearch(int[] a, int key) {
        int low = 0, high = a.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (a[mid] == key) {
                return mid;
            } else if (a[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 10));
    }
}
```

7. Input

Task 1: sorted array {10, 20, 30, 40, 50}, key = 40
Task 2: sorted array {10, 20, 30, 40, 50}, key = 35
Task 3: sorted array {10, 20, 30, 40, 50}, key = 10

8. Output

Task 1 output (actual run)

```
$ java Main.java
3
```

Task 2 output (actual run)

```
$ java Main.java
-1
```

Task 3 output (actual run)

```
$ java Main.java
0
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To search a sorted array by checking the middle value and discarding half of the array each time.

10. Viva-Voce Questions

Q1. What is the precondition for binary search?
Ans: The array must be sorted.

Q2. What is the time complexity?
Ans: O(log n).

Q3. Why use low + (high - low) / 2 for mid?
Ans: It avoids integer overflow that (low + high) / 2 could cause.

Q4. How many comparisons for 1,000,000 elements at most?
Ans: About 20, since log2(1,000,000) is roughly 20.

EXPERIMENT NO. 7

1. Experiment Title

Practice Set

2. Aim

To solve four short problems based on arrays, searching, sorting and loops.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

This set revisits core array techniques: linear search with a found flag or index, bubble sort, accumulating a product with a loop, and filtering values with an if statement using the remainder operator (value % 2 == 0 means even).

5. Algorithm / Procedure

Step 1: Task 1: scan {45, 52, 80}; if a[i] == 52 print its index and stop; otherwise print Not found.
Step 2: Task 2: bubble sort {3, 1, 2} and print it.
Step 3: Task 3: product = 1; multiply each element of {2, 3, 4} into product; print it.
Step 4: Task 4: for each element of {1, 2, 3, 4}, print it only if value % 2 == 0.

Time / Space Complexity: Task 1: O(n). Task 2: O(n^2). Tasks 3 and 4: O(n). Space O(1).

6. Program / Queries / Commands

Task 1: Search for 52 in {45, 52, 80}

```
public class Main {
    public static void main(String[] args) {
        int[] a = {45, 52, 80};
        int index = -1;
        for (int i = 0; i < a.length; i++) {
            if (a[i] == 52) {
                index = i;
                break;
            }
        }
        if (index != -1) {
            System.out.println("Found at index " + index);
        } else {
            System.out.println("Not found");
        }
    }
}
```

Task 2: Bubble sort {3, 1, 2}

```
public class Main {
    public static void main(String[] args) {
        int[] a = {3, 1, 2};
        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
            }
        }
        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Task 3: Product of {2, 3, 4}

```
public class Main {
    public static void main(String[] args) {
        int[] a = {2, 3, 4};
        int product = 1;
        for (int i = 0; i < a.length; i++) {
            product *= a[i];
        }
        System.out.println("Product: " + product);
    }
}
```

Task 4: Print even values of {1, 2, 3, 4}

```
public class Main {
    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4};
        for (int i = 0; i < a.length; i++) {
            if (a[i] % 2 == 0) {
                System.out.println(a[i]);
            }
        }
    }
}
```

7. Input

Task 1: array {45, 52, 80}
Task 2: array {3, 1, 2}
Task 3: array {2, 3, 4}
Task 4: array {1, 2, 3, 4}

8. Output

Task 1 output (actual run)

```
$ java Main.java
Found at index 1
```

Task 2 output (actual run)

```
$ java Main.java
1
2
3
```

Task 3 output (actual run)

```
$ java Main.java
Product: 24
```

Task 4 output (actual run)

```
$ java Main.java
2
4
```

9. Result

All 4 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To solve four short problems based on arrays, searching, sorting and loops.

10. Viva-Voce Questions

Q1. Why is product initialised to 1 and not 0?
Ans: 1 is the multiplicative identity; starting at 0 would make the product 0.

Q2. What does % return?
Ans: The remainder of integer division.

Q3. What does break do in the search loop?
Ans: It exits the loop once the key is found.

Q4. Which sort did you use and its complexity?
Ans: Bubble sort, O(n^2).

EXPERIMENT NO. 8

1. Experiment Title

Classes and Objects

2. Aim

To write a small class with fields and a method, and to create objects from it.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

A class is a blueprint that groups data (fields) and behaviour (methods). An object is an instance created with the new keyword; each object has its own copy of the fields. Methods can print data or return a value such as a boolean.

5. Algorithm / Procedure

Step 1: Define the class with its fields and method(s).
Step 2: In main, create objects with new and assign field values.
Step 3: Call the methods on the objects using the dot operator.
Step 4: Task 3: isLong() returns pages > 300.

Time / Space Complexity: All operations are O(1) time and space per object.

6. Program / Queries / Commands

Task 1: Student class with one object

```
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.roll = 5;
        s.name = "Rohan";
        s.show();
    }
}

class Student {
    int roll;
    String name;

    void show() {
        System.out.println("Roll: " + roll);
        System.out.println("Name: " + name);
    }
}
```

Task 2: Student class with two objects

```
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.roll = 5;
        s1.name = "Rohan";
        Student s2 = new Student();
        s2.roll = 8;
        s2.name = "Meera";
        s1.show();
        s2.show();
    }
}

class Student {
    int roll;
    String name;

    void show() {
        System.out.println("Roll: " + roll);
        System.out.println("Name: " + name);
    }
}
```

Task 3: Book class with isLong()

```
public class Main {
    public static void main(String[] args) {
        Book b1 = new Book();
        b1.pages = 450;
        Book b2 = new Book();
        b2.pages = 120;
        System.out.println(b1.isLong());
        System.out.println(b2.isLong());
    }
}

class Book {
    int pages;

    boolean isLong() {
        return pages > 300;
    }
}
```

7. Input

Task 1: none (values set in code)
Task 2: none (values set in code)
Task 3: none (values set in code)

8. Output

Task 1 output (actual run)

```
$ java Main.java
Roll: 5
Name: Rohan
```

Task 2 output (actual run)

```
$ java Main.java
Roll: 5
Name: Rohan
Roll: 8
Name: Meera
```

Task 3 output (actual run)

```
$ java Main.java
true
false
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To write a small class with fields and a method, and to create objects from it.

10. Viva-Voce Questions

Q1. What is the difference between a class and an object?
Ans: A class is a blueprint; an object is an instance of it.

Q2. What does the new keyword do?
Ans: It allocates memory and creates a new object.

Q3. Do two objects share field values?
Ans: No, each object has its own copy of instance fields.

Q4. What is the default value of an int field?
Ans: 0.

EXPERIMENT NO. 9

1. Experiment Title

Constructors

2. Aim

To set field values when an object is created, using constructors.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

A constructor is a special method with the same name as the class and no return type; it runs automatically when an object is created with new. A no-argument constructor sets default values, a parameterised constructor receives values, and this refers to the current object (useful when a parameter has the same name as a field).

5. Algorithm / Procedure

Step 1: Define the class with a field.
Step 2: Write a constructor: no-arg sets a fixed value; parameterised uses this.width = width.
Step 3: Task 3: constructor sets count = 0; increment() does count++.
Step 4: In main create the object(s) and print the field.

Time / Space Complexity: O(1) time and space per call.

6. Program / Queries / Commands

Task 1: No-argument constructor

```
public class Main {
    public static void main(String[] args) {
        Box b = new Box();
        System.out.println("Width: " + b.width);
    }
}

class Box {
    int width;

    Box() {
        width = 10;
    }
}
```

Task 2: Parameterised constructor using this

```
public class Main {
    public static void main(String[] args) {
        Box b = new Box(25);
        System.out.println("Width: " + b.width);
    }
}

class Box {
    int width;

    Box(int width) {
        this.width = width;
    }
}
```

Task 3: Counter class

```
public class Main {
    public static void main(String[] args) {
        Counter c = new Counter();
        c.increment();
        c.increment();
        c.increment();
        System.out.println("Count: " + c.count);
    }
}

class Counter {
    int count;

    Counter() {
        count = 0;
    }

    void increment() {
        count++;
    }
}
```

7. Input

Task 1: none
Task 2: width = 25
Task 3: increment() called 3 times

8. Output

Task 1 output (actual run)

```
$ java Main.java
Width: 10
```

Task 2 output (actual run)

```
$ java Main.java
Width: 25
```

Task 3 output (actual run)

```
$ java Main.java
Count: 3
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To set field values when an object is created, using constructors.

10. Viva-Voce Questions

Q1. What is a constructor?
Ans: A special method that initialises an object when it is created.

Q2. Can a constructor have a return type?
Ans: No.

Q3. What is the use of the this keyword?
Ans: It refers to the current object and separates fields from same-named parameters.

Q4. What is a default constructor?
Ans: The no-argument constructor Java supplies when none is written.

EXPERIMENT NO. 10

1. Experiment Title

Parent and Child Classes

2. Aim

To make one class extend another so that the child class uses the parent's fields and methods.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

Inheritance lets a child (sub) class acquire the fields and methods of a parent (super) class using the extends keyword. The child can use them without rewriting code. If the parent has a constructor with parameters, the child constructor must call it with super(...) as its first statement.

5. Algorithm / Procedure

Step 1: Define the parent class with its field/method/constructor.
Step 2: Define the child class with extends (and super(name) if needed).
Step 3: In main create a child object and call inherited methods or read inherited fields.
Step 4: Task 3: set name directly and call greet().

Time / Space Complexity: O(1) time and space per call.

6. Program / Queries / Commands

Task 1: Dog extends Animal

```
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}

class Animal {
    void sound() {
        System.out.println("Some sound");
    }
}

class Dog extends Animal {
}
```

Task 2: Student extends Person using super(name)

```
public class Main {
    public static void main(String[] args) {
        Student s = new Student("Neha");
        System.out.println("Name: " + s.name);
    }
}

class Person {
    String name;

    Person(String name) {
        this.name = name;
    }
}

class Student extends Person {
    Student(String name) {
        super(name);
    }
}
```

Task 3: Teacher extends Person with greet()

```
public class Main {
    public static void main(String[] args) {
        Teacher t = new Teacher();
        t.name = "Kiran";
        t.greet();
    }
}

class Person {
    String name;

    void greet() {
        System.out.println("Hello, " + name);
    }
}

class Teacher extends Person {
}
```

7. Input

Task 1: none
Task 2: name = Neha
Task 3: name = Kiran

8. Output

Task 1 output (actual run)

```
$ java Main.java
Some sound
```

Task 2 output (actual run)

```
$ java Main.java
Name: Neha
```

Task 3 output (actual run)

```
$ java Main.java
Hello, Kiran
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To make one class extend another so that the child class uses the parent's fields and methods.

10. Viva-Voce Questions

Q1. Which keyword is used for inheritance in Java?
Ans: extends.

Q2. What is the purpose of super()?
Ans: It calls the parent class constructor.

Q3. Does Java support multiple inheritance of classes?
Ans: No, a class can extend only one class.

Q4. What does the child class inherit?
Ans: The accessible fields and methods of the parent.

EXPERIMENT NO. 11

1. Experiment Title

Method Overriding

2. Aim

To change a method in a child class so that it gives a different answer from the parent version.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

Method overriding occurs when a child class defines a method with the same name and signature as one in its parent. At run time Java calls the child's version, even when the object is stored in a parent-type variable (runtime polymorphism).

5. Algorithm / Procedure

Step 1: Define Shape with area() returning 0.
Step 2: Define Square/Rectangle extending Shape and overriding area().
Step 3: In main create the object and set its fields.
Step 4: Call area() and print; in Task 3 use a Shape reference to a Square object.

Time / Space Complexity: O(1) time and space.

6. Program / Queries / Commands

Task 1: Square overrides area()

```
public class Main {
    public static void main(String[] args) {
        Square s = new Square();
        s.side = 4;
        System.out.println("Area: " + s.area());
    }
}

class Shape {
    int area() {
        return 0;
    }
}

class Square extends Shape {
    int side;

    @Override
    int area() {
        return side * side;
    }
}
```

Task 2: Rectangle overrides area()

```
public class Main {
    public static void main(String[] args) {
        Rectangle r = new Rectangle();
        r.length = 4;
        r.breadth = 5;
        System.out.println("Area: " + r.area());
    }
}

class Shape {
    int area() {
        return 0;
    }
}

class Rectangle extends Shape {
    int length;
    int breadth;

    @Override
    int area() {
        return length * breadth;
    }
}
```

Task 3: Runtime polymorphism with a Shape variable

```
public class Main {
    public static void main(String[] args) {
        Shape s = new Square(3);
        System.out.println("Area: " + s.area());
    }
}

class Shape {
    int area() {
        return 0;
    }
}

class Square extends Shape {
    int side;

    Square(int side) {
        this.side = side;
    }

    @Override
    int area() {
        return side * side;
    }
}
```

7. Input

Task 1: side = 4
Task 2: length = 4, breadth = 5
Task 3: side = 3

8. Output

Task 1 output (actual run)

```
$ java Main.java
Area: 16
```

Task 2 output (actual run)

```
$ java Main.java
Area: 20
```

Task 3 output (actual run)

```
$ java Main.java
Area: 9
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To change a method in a child class so that it gives a different answer from the parent version.

10. Viva-Voce Questions

Q1. What is method overriding?
Ans: Redefining a parent method in a child class with the same signature.

Q2. What is the use of @Override?
Ans: It makes the compiler check that a parent method is really being overridden.

Q3. Which version runs for Shape s = new Square(3)?
Ans: Square's area(), decided at run time.

Q4. How does overriding differ from overloading?
Ans: Overriding keeps the signature and is in a child class; overloading changes the parameters in the same class.

EXPERIMENT NO. 12

1. Experiment Title

Recursive Factorial, Sum and Power

2. Aim

To write methods that call themselves on a smaller input until a base case is reached.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

Recursion solves a problem by reducing it to a smaller instance of itself. Every recursive method needs a base case that stops the calls and a recursive case that moves towards it. Calls are stored on the call stack and return in reverse order.

5. Algorithm / Procedure

Step 1: factorial(n): if n == 0 return 1; else return n * factorial(n - 1).
Step 2: sum(n): if n == 0 return 0; else return n + sum(n - 1).
Step 3: power(base, exp): if exp == 0 return 1; else return base * power(base, exp - 1).
Step 4: Print factorial(5), sum(10) and power(2, 5).

Time / Space Complexity: All three: O(n) time (O(exp) for power) and O(n) stack space.

6. Program / Queries / Commands

Task 1: Recursive factorial

```
public class Main {
    static int factorial(int n) {
        if (n == 0) {
            return 1;
        }
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(factorial(5));
    }
}
```

Task 2: Recursive sum of 1 to n

```
public class Main {
    static int sum(int n) {
        if (n == 0) {
            return 0;
        }
        return n + sum(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(sum(10));
    }
}
```

Task 3: Recursive power

```
public class Main {
    static int power(int base, int exp) {
        if (exp == 0) {
            return 1;
        }
        return base * power(base, exp - 1);
    }

    public static void main(String[] args) {
        System.out.println(power(2, 5));
    }
}
```

7. Input

Task 1: n = 5
Task 2: n = 10
Task 3: base = 2, exp = 5

8. Output

Task 1 output (actual run)

```
$ java Main.java
120
```

Task 2 output (actual run)

```
$ java Main.java
55
```

Task 3 output (actual run)

```
$ java Main.java
32
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To write methods that call themselves on a smaller input until a base case is reached.

10. Viva-Voce Questions

Q1. What is a base case?
Ans: The condition that stops the recursion.

Q2. What happens without a base case?
Ans: Infinite recursion leading to StackOverflowError.

Q3. What is 0! by definition?
Ans: 1.

Q4. What is the space complexity of recursive factorial?
Ans: O(n) because of n stacked calls.

EXPERIMENT NO. 13

1. Experiment Title

Recursion on Arrays

2. Aim

To walk through an array with an index that moves one step further in each recursive call.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

To process an array recursively, the method receives the array and a current index i. The base case is when i reaches the end (or 0 when going backwards). The recursive case handles element a[i] and calls itself with i + 1 (or i - 1).

5. Algorithm / Procedure

Step 1: sum(a, i): if i == a.length return 0; else return a[i] + sum(a, i + 1).
Step 2: max(a, i): if i == a.length - 1 return a[i]; else return larger of a[i] and max(a, i + 1).
Step 3: printReverse(a, i): if i < 0 return; print a[i]; call printReverse(a, i - 1).

Time / Space Complexity: All: O(n) time and O(n) stack space.

6. Program / Queries / Commands

Task 1: Recursive array sum

```
public class Main {
    static int sum(int[] a, int i) {
        if (i == a.length) {
            return 0;
        }
        return a[i] + sum(a, i + 1);
    }

    public static void main(String[] args) {
        int[] a = {3, 6, 9, 12, 15};
        System.out.println(sum(a, 0));
    }
}
```

Task 2: Recursive array maximum

```
public class Main {
    static int max(int[] a, int i) {
        if (i == a.length - 1) {
            return a[i];
        }
        int rest = max(a, i + 1);
        return a[i] > rest ? a[i] : rest;
    }

    public static void main(String[] args) {
        int[] a = {3, 6, 9, 12, 15};
        System.out.println(max(a, 0));
    }
}
```

Task 3: Print array in reverse

```
public class Main {
    static void printReverse(int[] a, int i) {
        if (i < 0) {
            return;
        }
        System.out.println(a[i]);
        printReverse(a, i - 1);
    }

    public static void main(String[] args) {
        int[] a = {3, 6, 9, 12, 15};
        printReverse(a, a.length - 1);
    }
}
```

7. Input

Task 1: array {3, 6, 9, 12, 15}, i = 0
Task 2: array {3, 6, 9, 12, 15}, i = 0
Task 3: array {3, 6, 9, 12, 15}, i = 4

8. Output

Task 1 output (actual run)

```
$ java Main.java
45
```

Task 2 output (actual run)

```
$ java Main.java
15
```

Task 3 output (actual run)

```
$ java Main.java
15
12
9
6
3
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To walk through an array with an index that moves one step further in each recursive call.

10. Viva-Voce Questions

Q1. What is the base case for the recursive sum?
Ans: i == a.length, returning 0.

Q2. Why pass the index as a parameter?
Ans: To track the current position without copying the array.

Q3. How does printReverse stop?
Ans: When i becomes less than 0.

Q4. Can these be written iteratively?
Ans: Yes, with a for loop and without stack usage.

EXPERIMENT NO. 14

1. Experiment Title

Sorted Check, Search and Fibonacci

2. Aim

To use recursion to check an array, find a value by index, and compute Fibonacci numbers.

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

isSorted checks a[i] <= a[i + 1] and recurses on the rest. findIndex checks a[i] against the key and recurses on i + 1. The Fibonacci sequence is defined by fib(0) = 0, fib(1) = 1 and fib(n) = fib(n - 1) + fib(n - 2); the plain recursion makes two calls each time, which gives exponential running time.

5. Algorithm / Procedure

Step 1: isSorted(a, i): if i >= a.length - 1 return true; if a[i] > a[i + 1] return false; return isSorted(a, i + 1).
Step 2: findIndex(a, key, i): if i == a.length return -1; if a[i] == key return i; return findIndex(a, key, i + 1).
Step 3: fib(n): if n <= 1 return n; return fib(n - 1) + fib(n - 2).
Step 4: Print the results for the given inputs.

Time / Space Complexity: isSorted and findIndex: O(n) time, O(n) stack. fib: O(2^n) time, O(n) stack.

6. Program / Queries / Commands

Task 1: Recursive isSorted

```
public class Main {
    static boolean isSorted(int[] a, int i) {
        if (i >= a.length - 1) {
            return true;
        }
        if (a[i] > a[i + 1]) {
            return false;
        }
        return isSorted(a, i + 1);
    }

    public static void main(String[] args) {
        int[] a = {2, 4, 6, 8, 10};
        System.out.println(isSorted(a, 0));
    }
}
```

Task 2: Recursive findIndex

```
public class Main {
    static int findIndex(int[] a, int key, int i) {
        if (i == a.length) {
            return -1;
        }
        if (a[i] == key) {
            return i;
        }
        return findIndex(a, key, i + 1);
    }

    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(findIndex(a, 80, 0));
    }
}
```

Task 3: Recursive Fibonacci

```
public class Main {
    static int fib(int n) {
        if (n <= 1) {
            return n;
        }
        return fib(n - 1) + fib(n - 2);
    }

    public static void main(String[] args) {
        System.out.println(fib(7));
    }
}
```

7. Input

Task 1: array {2, 4, 6, 8, 10}, i = 0
Task 2: array {45, 67, 80, 52, 90}, key = 80, i = 0
Task 3: n = 7

8. Output

Task 1 output (actual run)

```
$ java Main.java
true
```

Task 2 output (actual run)

```
$ java Main.java
2
```

Task 3 output (actual run)

```
$ java Main.java
13
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To use recursion to check an array, find a value by index, and compute Fibonacci numbers.

10. Viva-Voce Questions

Q1. What are the first eight Fibonacci numbers?
Ans: 0, 1, 1, 2, 3, 5, 8, 13.

Q2. Why is recursive Fibonacci slow?
Ans: It recomputes the same values many times, giving O(2^n) time.

Q3. How can it be made faster?
Ans: Memoization or an iterative loop gives O(n).

Q4. When does isSorted return true?
Ans: When no neighbouring pair is out of order, or when 0 or 1 elements remain.

EXPERIMENT NO. 15

1. Experiment Title

Subsets and Arrangements

2. Aim

To build answers one choice at a time, print them, and undo each choice before trying the next one (backtracking).

3. Software / Tools Required

Java (JDK 17 or later); any IDE or editor (VS Code, IntelliJ IDEA, Eclipse) or the command line (`java Main.java`).

4. Theory

Backtracking explores all possibilities by making a choice, recursing, then undoing the choice. For subsets, each element is either skipped or included, giving 2^n subsets. For permutations, each position takes any unused letter; a used[] array marks letters in use, and the mark is removed on return, giving n! arrangements.

5. Algorithm / Procedure

Step 1: Subsets: solve(i): if i == n print the current subset; else skip a[i] and recurse; include a[i], recurse, then remove it.
Step 2: Permutations: solve(pos): if pos == n print the arrangement; else for each letter k not used: mark used[k], set cur[pos] = letter, recurse on pos + 1, unmark used[k].
Step 3: Print each result on its own line.

Time / Space Complexity: Subsets: O(n * 2^n) time. Permutations: O(n * n!) time. Space O(n) for the recursion.

6. Program / Queries / Commands

Task 1: All subsets of {1, 2, 3}

```
import java.util.ArrayList;

public class Main {
    static void subsets(int[] a, int i, ArrayList<Integer> cur) {
        if (i == a.length) {
            StringBuilder sb = new StringBuilder("{");
            for (int k = 0; k < cur.size(); k++) {
                if (k > 0) {
                    sb.append(", ");
                }
                sb.append(cur.get(k));
            }
            sb.append("}");
            System.out.println(sb);
            return;
        }
        subsets(a, i + 1, cur);          // skip
        cur.add(a[i]);                   // include
        subsets(a, i + 1, cur);
        cur.remove(cur.size() - 1);      // undo
    }

    public static void main(String[] args) {
        subsets(new int[]{1, 2, 3}, 0, new ArrayList<Integer>());
    }
}
```

Task 2: All arrangements of a, b, c

```
public class Main {
    static void permute(char[] letters, boolean[] used, char[] cur, int pos) {
        if (pos == letters.length) {
            System.out.println(new String(cur));
            return;
        }
        for (int k = 0; k < letters.length; k++) {
            if (!used[k]) {
                used[k] = true;
                cur[pos] = letters[k];
                permute(letters, used, cur, pos + 1);
                used[k] = false;     // undo
            }
        }
    }

    public static void main(String[] args) {
        char[] letters = {'a', 'b', 'c'};
        permute(letters, new boolean[letters.length], new char[letters.length], 0);
    }
}
```

Task 3: All arrangements of x, y

```
public class Main {
    static void permute(char[] letters, boolean[] used, char[] cur, int pos) {
        if (pos == letters.length) {
            System.out.println(new String(cur));
            return;
        }
        for (int k = 0; k < letters.length; k++) {
            if (!used[k]) {
                used[k] = true;
                cur[pos] = letters[k];
                permute(letters, used, cur, pos + 1);
                used[k] = false;     // undo
            }
        }
    }

    public static void main(String[] args) {
        char[] letters = {'x', 'y'};
        permute(letters, new boolean[letters.length], new char[letters.length], 0);
    }
}
```

7. Input

Task 1: array {1, 2, 3}
Task 2: letters a, b, c
Task 3: letters x, y

8. Output

Task 1 output (actual run)

```
$ java Main.java
{}
{3}
{2}
{2, 3}
{1}
{1, 3}
{1, 2}
{1, 2, 3}
```

Task 2 output (actual run)

```
$ java Main.java
abc
acb
bac
bca
cab
cba
```

Task 3 output (actual run)

```
$ java Main.java
xy
yx
```

9. Result

All 3 programs of this experiment were compiled and executed successfully and produced the expected output. Aim achieved: To build answers one choice at a time, print them, and undo each choice before trying the next one (backtracking).

10. Viva-Voce Questions

Q1. What is backtracking?
Ans: Trying a choice, recursing, and undoing the choice to explore other options.

Q2. How many subsets does a set of n elements have?
Ans: 2^n.

Q3. How many arrangements of n distinct letters?
Ans: n!.

Q4. Why must used[k] be reset to false?
Ans: So that the letter can be used in other arrangements.